

SpaceScout: design note

1. Approach

A search request goes through six steps, each with one job:

```
parse (LLM, rules fallback) → normalise (code) → decide: search or clarify (code)
→ match hard constraints (code) → rank (code) → explain (code facts, optional LLM phrasing)
```

The core decision is **what the LLM is allowed to do**. It turns free text into a fixed schema. It never sees the listings, never computes anything, and never decides what is shown. That one rule is what makes the honesty requirement enforceable, not just hoped for.

This is deliberately not an agent: the flow is fixed and each step runs once, so an agent loop would add latency and failure modes without adding capability.

2. Splitting work between the LLM and regular code

Concern	Owner	Reasoning
Parsing	LLM	“4 of us”, “kal dopahar”, “meetng room fr 6”, “ideally a whiteboard” is a language problem, and it is exactly where the rule-based parser fails in the evaluation. A strict JSON schema limits amenities to 15 known keys; anything else becomes “Can’t filter on: sea view” instead of silence. The output is re-validated, with one repair retry, then a fallback to the rule parser.
Normalisation	Code	The LLM outputs "tomorrow" + "afternoon", not dates; code produces <i>Tue 06 Oct, 12:00–17:00, 2-hour slot</i> and shows the assumption. The same goes for location aliases, typos, “near X” (areas within 5 km) and budget units: ₹1,800/hr for 4 people is ₹450 per person. That is arithmetic, not language.
Filtering	Code	Hard constraints (area, capacity, budget, time, space type, “must-have” amenities) are checked against every listing. Each miss is recorded as a costed violation, for example “₹450/person/hr: ₹150 over your ₹300/person/hr” or “In Khar, 1.6 km from Bandra”.
Ranking	Code	Explicit weights (below), deterministic, and each result’s score breakdown is shown in the UI (“Why this rank”).
Explanation	Code builds a fact sheet; the LLM may rephrase it	The template alone meets the requirement: “ <i>Quiet space; has whiteboard; seats 4, enough for your 4. Trade-off: Wi-Fi is only 50 Mbps.</i> ” When the LLM rewrites it, each sentence is checked. Any number, amenity or noise word not in that listing’s facts gets the text rejected and the template used. The explainer never receives the user’s raw text, so a prompt injection in the query cannot reach it.

Checking the LLM against the user’s words (`reconcile.py`, added after testing live). The model sometimes made a request *stricter* than stated:

- it invented a calendar date for “sunday”;
- it read “under ₹1,200 an hour” as per person;
- it decided a “client pitch” needs a meeting room.

Each would silently hide valid listings. So any hard constraint the LLM sets must be backed by the text: an explicit date, a “per person” or “per day” phrase, a named space type. If it isn’t, the constraint is relaxed and shown as an assumption. The rule: the LLM may interpret the request, but it may not tighten it.

Hard vs soft. Location, capacity, budget, time and must-have amenities are hard. “Ideally”, “preferably” and qualities like “quiet” or “fast wifi” are soft: they affect rank, never eligibility. The UI’s “What I understood” panel groups them into must match, preferences, assumptions and can’t filter on.

Messy cases. A vague request gets a clarifying question, plus three well-reviewed spaces labelled as not tailored. Off-topic requests and cities we don’t cover are declined with a question, and nothing is searched. When nothing matches, the closest alternatives are shown, ordered by how badly they miss and each stating the miss. The single relaxation that unlocks the most options is reported, for example *“the cheapest is ₹450/person/hr against your ₹300”* or *“the largest space fits 40”*. With fewer than three exact matches, labelled alternatives are added.

3. Ranking, from a marketplace’s point of view

$\text{score} = 0.45 \cdot \text{fit} + 0.25 \cdot \text{trust} + 0.15 \cdot \text{value} + 0.15 \cdot \text{conversion}$ (with no soft preferences, fit is dropped and the other three are re-weighted).

- **Fit:** the share of soft preferences met.
- **Trust:** a Bayesian-average rating (10 pseudo-reviews at the market mean), so 5.0★ from 3 reviews ranks below 4.8★ from 96.
- **Value:** price against the user’s budget. It is weighted low on purpose: bookers don’t want cheapest-first, but ignoring price drifts the top results upmarket.
- **Conversion:** instant-book, and a right-sized room. Four people in a 20-seat room is a poor experience and a likely cancellation.

There is no paid boosting. A sponsored slot would be a separate, labelled row.

4. Model choice

Default: openai/gpt-oss-20b **served by Groq**, through any OpenAI-compatible endpoint (LLM_BASE_URL, LLM_MODEL).

- **Guaranteed schema.** Groq supports *strict* JSON-schema output for this model (constrained decoding), so malformed output is a provider bug, not a normal case. If a provider rejects the schema, the client drops to JSON mode, and validation and fallback still apply.
- **Size fits the task.** Extracting a 19-field schema doesn’t need a frontier model. A small model on fast hardware answers in about 0.5–1.3 s (measured on the deployed app).
- **Replaceable.** Only `llm_client.py` knows about HTTP; switching provider is a config change.

5. What changes at 100,000 listings

Today every listing is evaluated in Python on each request: fine for 40 listings, wrong for 100,000.

1. **Filter in the database.** Move to Postgres with indexes on area, type, capacity and price, and PostGIS for “near Bandra” as a radius query. Hard constraints become a WHERE clause returning hundreds of candidates, not 100k rows.
2. **Availability from the booking system.** Query a calendar service for the candidates only, with a short-TTL cache, and always re-check at booking time.
3. **Relaxation as facet counts.** “What if budget were dropped?” becomes one COUNT per constraint, or a search-engine facet, instead of re-running the matcher.
4. **Two-stage ranking.** SQL retrieves roughly the top 500; a learned ranker trained on searches, clicks and completed bookings re-orders them. The hand-set weights become features and a baseline.
5. **Embeddings earn a place.** Preferences like “good for podcast recording” can’t map to 15 amenity keys. Embed listing descriptions and reviews (pgvector is enough at this size) and use similarity as a ranking feature, never as a filter, so it can’t override a hard constraint.
6. **Cost and quality at scale.** The LLM never sees listings, so its cost doesn’t grow with inventory; budget per search and use a paid tier. Run labelled production queries in CI on every prompt or model change, and track zero-result rate and search-to-booking conversion.